

10/22/2025

PROJECT REPORT

SOCIAL MEDIA POSTS ON SENTIMENT ANALYSIS

Submitted To,
Name of Trainer:
Swathy ma'am

Submitted By,
Name: Nihal Husain.T
Reg No:1805229
Course: MDAD
Email: nihalhusain666@gmail.com

TABLE OF CONTENTS

S. No.	Section Title
1	About the Dataset
2	Column Descriptions
3	Introduction
4	Data Preprocessing
5	Exploratory Data Analysis (EDA)
6	Text Cleaning and Preprocessing
8	Feature Extraction (TF-IDF/Bag of Words)
11	Conclusion

INTRODUCTION

- Social media has become one of the most influential communication platforms, where millions of users express their opinions, emotions, and feedback daily. Sentiment analysis — also known as opinion mining — is a natural language processing (NLP) technique used to identify and categorize opinions expressed in text to determine whether the sentiment is positive, negative, or neutral.
- This project focuses on analyzing the sentiments of social media posts. By applying machine learning and NLP techniques, the aim is to automatically classify user-generated content and gain insights into public opinion trends. Such analysis can help businesses, researchers, and organizations understand customer satisfaction, brand perception, and emerging issues on social media.

ABOUT THE DATASET

- The dataset used in this project contains textual data from social media posts along with their associated sentiment labels.

Typical columns include:

1.Text - the actual content of a user's post.

2.Sentiment/Label - the sentiment class (e.g., Positive, Negative, or Neutral).

- Before training, several data preprocessing steps were performed:
 - 1.Removal of unwanted characters, punctuation, URLs, and stopwords.
 - 2.Conversion of text to lowercase.
 - 3.Tokenization and lemmatization/stemming.
 - 4.Transformation into numerical format using TF-IDF or CountVectorizer.

COLUMN DESCRIPTION

Column Name	Description
Year	The year when the tweet was posted (e.g., 2018)..
Month	The month of the tweet (numeric, e.g., 8 for August).
Day	The day of the month the tweet was posted.
Time of Tweet	Indicates the time of day when the tweet was made (e.g., “morning”, “afternoon”, “evening”, “night”).
Text	The actual tweet or post content being analyzed.
Sentiment	The sentiment classification of the tweet — likely one of “positive”, “negative”, or “neutral”.
Platform	The social media platform where the post was made (e.g., “Twitter”).

LOADING LIBRARIES

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
```

In [117]:

```

import string
from wordcloud import WordCloud
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# 2. Download necessary NLTK resources
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
import warnings
warnings.filterwarnings('ignore')
[nltk_data] Downloading package punkt to
[nltk_data]      C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]      C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]      C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

```

DATASET STRUCTURE AND QUALITY ASSESSMENT

```

In [120]: df=pd.read_csv(r"C:\Users\NIHAL\Downloads\sentiment_analysis.csv")

In [122]: df.head()

```

Out[122]:

	Year	Month	Day	Time of Tweet	text	sentiment	Platform
0	2018	8	18	morning	What a great day!!! Looks like dream.	positive	Twitter
1	2018	8	18	noon	I feel sorry, I miss you here in the sea beach	positive	Facebook
2	2017	8	18	night	Don't angry me	negative	Facebook

	Year	Month	Day	Time of Tweet	text	sentiment	Platform
3	2022	6	8	morning	We attend in the class just for listening teac...	negative	Facebook
4	2022	6	8	noon	Those who want to go, let them go	negative	Instagram

```
df.size
```

In [124]:

```
3493
```

Out[124]:

```
df.shape
```

In [126]:

```
(499, 7)
```

Out[126]:

In [15]:

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 499 entries, 0 to 498
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Year            499 non-null    int64
 1   Month           499 non-null    int64
 2   Day             499 non-null    int64
 3   Time of Tweet   499 non-null    object
 4   text            499 non-null    object
 5   sentiment       499 non-null    object
 6   Platform        499 non-null    object
dtypes: int64(3), object(4)
memory usage: 27.4+ KB
```

```
df.describe().T
```

In [17]:

Out[17]:

	count	mean	std	min	25%	50%	75%	max
Year	499.0	2020.394790	2.830991	2010.0	2019.0	2021.0	2023.0	2023.0
Month	499.0	6.084168	3.652711	1.0	2.0	6.0	9.0	12.0
Day	499.0	15.693387	8.804673	1.0	8.0	15.0	22.0	31.0

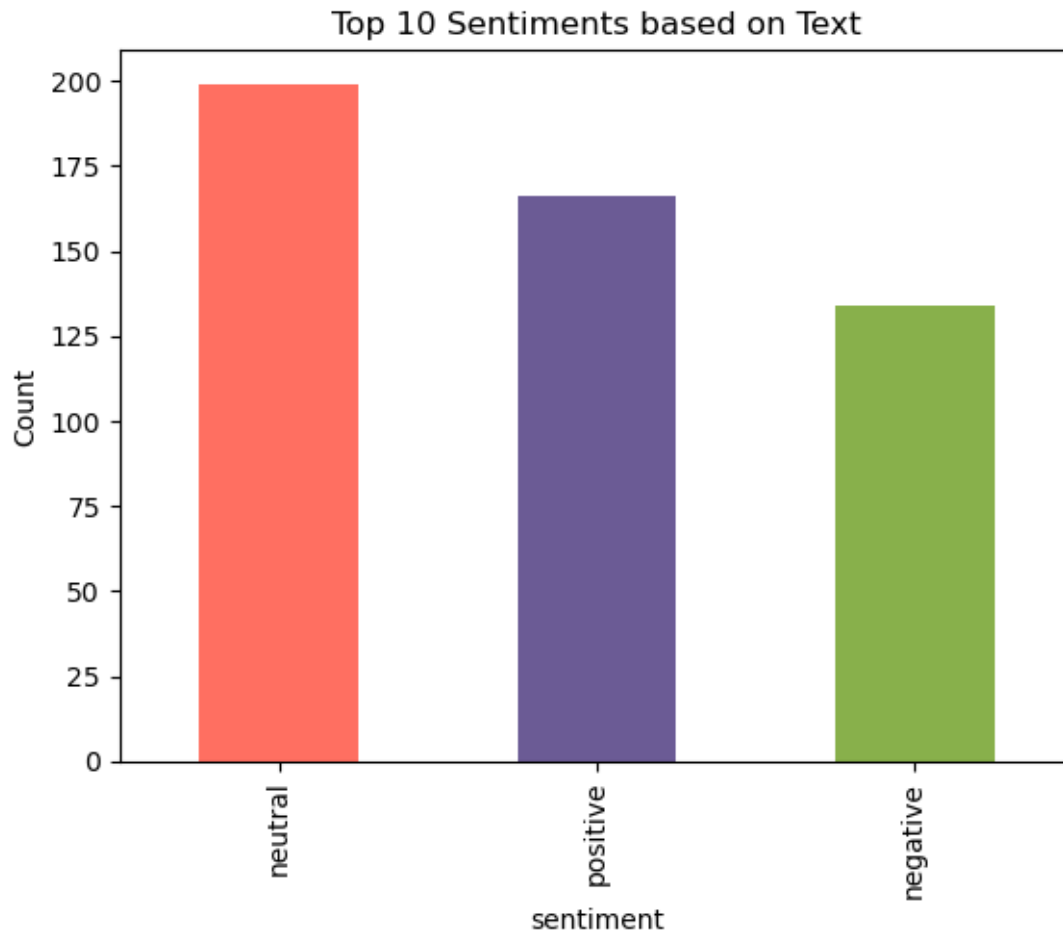
EDA

Sentiment Distribution :

In [21]:

```
colors = ['#FF6F61', '#6B5B95', '#88B04B', '#F7CAC9', '#92A8D1',
          '#955251', '#B565A7', '#009B77', '#DD4124', '#45B8AC']
```

```
df['sentiment'].value_counts().nlargest(10).plot(kind='bar', color=colors)
plt.title('Top 10 Sentiments based on Text')
plt.xlabel('sentiment')
plt.ylabel('Count')
plt.show()
```



In [23]:

```
df['Platform'].value_counts()
```

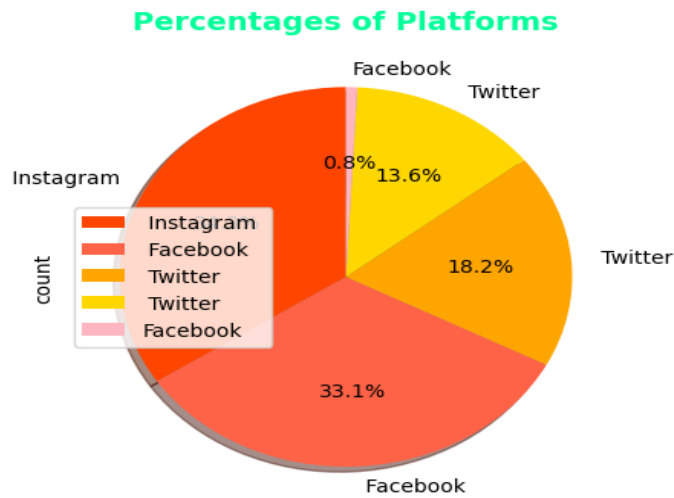
Out[23]:

```
Platform
Instagram    171
Facebook     165
Twitter       91
Twitter       68
Facebook       4
Name: count, dtype: int64
```

In [25]:

```
colors = ['#FF4500', '#FF6347', '#FFA500', '#FFD700', '#FFB6C1']
df['Platform'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    colors=colors,
    startangle=90,
    shadow=True
)
```

```
plt.title('Percentages of Platforms', color='#00FF99', fontsize=14, fontweight='bold')
plt.legend()
plt.show()
```



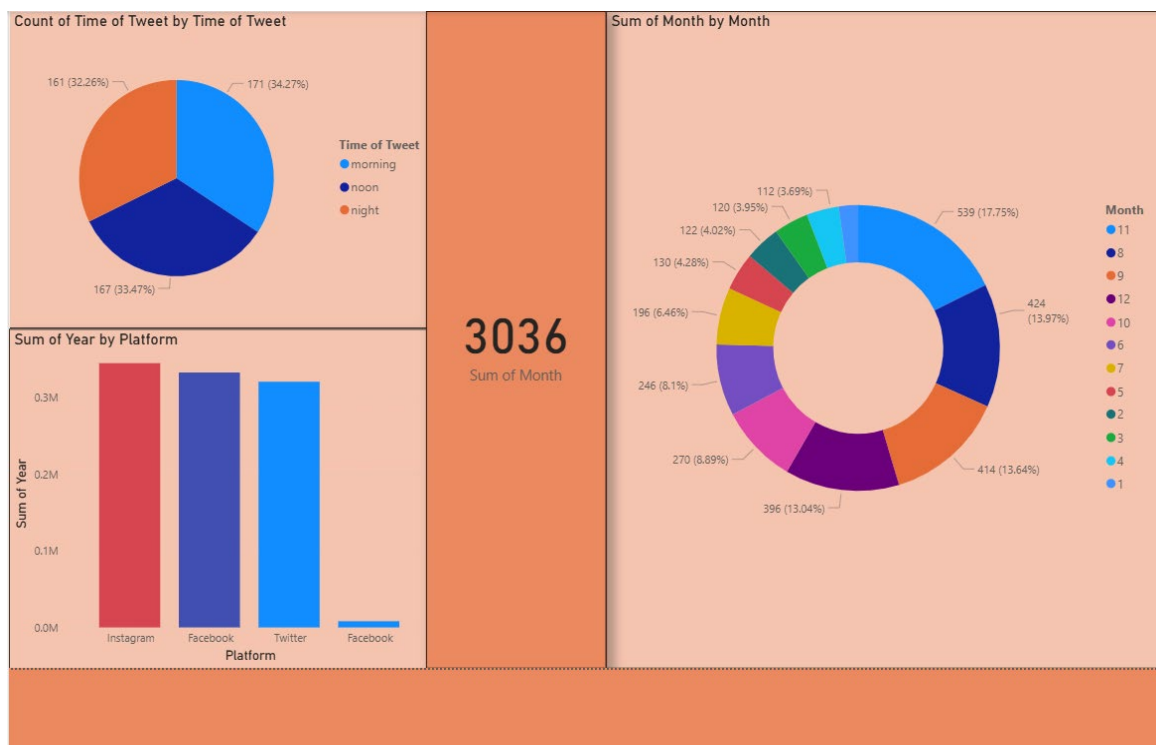
In [29]:

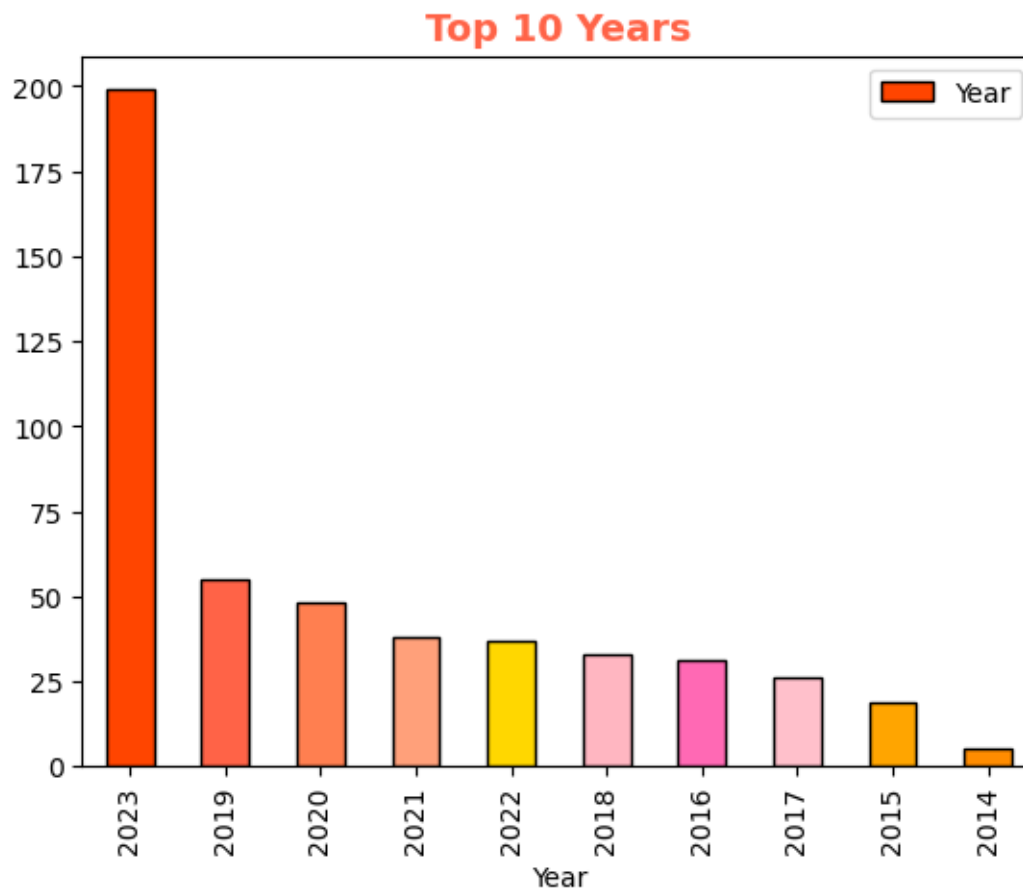
```
colors = ['#FF4500', '#FF6347', '#FF7F50', '#FFA07A', '#FFD700',
          '#FFB6C1', '#FF69B4', '#FFC0CB', '#FFA500', '#FF8C00']

df['Year'].value_counts().nlargest(10).plot(
    kind='bar',
    color=colors,
    edgecolor='black'
)

plt.title('Top 10 Years', color='#FF6347', fontsize=14, fontweight='bold')
plt.legend(['Year'])
plt.show()
```

POWER-BI DASHBOARD





In [31]:

```
df.columns
```

Out[31]:

```
Index(['Year', 'Month', 'Day', 'Time of Tweet', 'text', 'sentiment',
      'Platform'],
      dtype='object')
```

In [33]:

```
df = df.drop(['Year', 'Month', 'Day', 'Time of Tweet', 'Platform'], axis=1)
df
```

Out[33]:

	text	sentiment
0	What a great day!!! Looks like dream.	positive
1	I feel sorry, I miss you here in the sea beach	positive
2	Don't angry me	negative
3	We attend in the class just for listening teac...	negative
4	Those who want to go, let them go	negative
...	...	...
494	According to , a quarter of families under six...	negative
495	the plan to not spend money is not going well	negative
496	uploading all my bamboozle pictures of facebook	neutral
497	congratulations ! you guys finish a month ear...	positive
498	actually, I wish I was back in Tahoe. I miss...	negative

499 rows × 2 columns

PREPROCESSING TEXT DATA

In [35]:

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\NIHAL\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Out[35]:

True

In [39]:

```
df['text'] = df['text'].astype(str)
```

In [41]:

```
def wordopt(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+|https\S+", '', text)
    text = re.sub(r'\@w+|\#','', text)
    text = re.sub(r'^A-Za-z\s','', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

In [43]:

```
df['text'] = df['text'].apply(wordopt)
```

In [45]:

```
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))
```

In [47]:

```
def preprocess_text(text):
    words = word_tokenize(text)
    words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return ' '.join(words)
```

In [51]:

```
df['text'] = df['text'].apply(preprocess_text)
```

In [54]:

```
print("✅ Text preprocessing complete!")
print(df[['text', 'text']].head())
```

✅ Text preprocessing complete!

```

text \
0      great day look like dream
1      feel sorry miss sea beach
2      dont angry
3 attend class listening teacher reading slide n...
```

```

4                                want go let go

                                text
0                                great day look like dream
1                                feel sorry miss sea beach
2                                dont angry
3 attend class listening teacher reading slide n...
4                                want go let go

```

WORD CLOUD

In [57]:

```

# Step 1: Make sure pandas is imported and your DataFrame exists
import pandas as pd
from wordcloud import WordCloud
import matplotlib.pyplot as plt

all_reviews = " ".join(df['text'].dropna().astype(str).tolist())

# Step 2: Generate the WordCloud
wc = WordCloud(
    max_words=400,
    width=1600,
    height=800,
    background_color='white',
    colormap='Blues',      # clean blue tones
    contour_color='#004080',
    contour_width=2,
    prefer_horizontal=1.0
).generate(all_reviews)

# Step 3: Display the WordCloud
plt.figure(figsize=(20, 10))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off') # remove axes
plt.show()

```



```

4         negative
      ...
494     negative
495     negative
496     neutral
497     positive
498     negative
Name: sentiment, Length: 499, dtype: object

```

In [62]:

```

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=42)

```

In [64]:

```

print(x_train.shape,x_test.shape,y_train.shape,y_test.shape)
(374,) (125,) (374,) (125,)

```

NATURAL LANGUAGE PROCESSING

In [67]:

```

vectorization=TfidfVectorizer()
xv_train=vectorization.fit_transform(x_train)
xv_test=vectorization.transform(x_test)

```

In [69]:

```

xv_train

```

Out[69]:

```

<374x1059 sparse matrix of type '<class 'numpy.float64'>'
  with 2352 stored elements in Compressed Sparse Row format>

```

In [71]:

```

xv_train.shape

```

Out[71]:

```

(374, 1059)

```

In [73]:

```

xv_test.shape

```

Out[73]:

```

(125, 1059)

```

NAIVE BAYES THEOREM

MultinomialNB

In [75]:

```

modell = MultinomialNB()
modell.fit(xv_train,y_train)
predict1 = modell.predict(xv_test)
predict1

```

Out[75]:

```

array(['positive', 'neutral', 'neutral', 'neutral', 'neutral', 'negative',
       'neutral', 'neutral', 'neutral', 'neutral', 'negative', 'positive',
       'neutral', 'positive', 'positive', 'positive', 'positive',

```

```

'positive', 'positive', 'positive', 'positive', 'neutral',
'neutral', 'positive', 'positive', 'negative', 'positive',
'neutral', 'positive', 'negative', 'neutral', 'neutral',
'positive', 'negative', 'neutral', 'negative', 'neutral',
'positive', 'positive', 'neutral', 'neutral', 'neutral',
'positive', 'neutral', 'positive', 'positive', 'neutral',
'positive', 'neutral', 'neutral', 'positive', 'neutral', 'neutral',
'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'positive',
'neutral', 'neutral', 'positive', 'positive', 'neutral',
'positive', 'negative', 'negative', 'neutral', 'positive',
'neutral', 'neutral', 'neutral', 'neutral', 'negative', 'neutral',
'neutral', 'positive', 'negative', 'negative', 'neutral',
'negative', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
'neutral', 'neutral', 'negative', 'negative', 'neutral',
'positive', 'neutral', 'neutral', 'neutral', 'positive', 'neutral',
'neutral', 'neutral', 'neutral', 'neutral', 'positive', 'positive',
'positive', 'positive', 'positive', 'neutral', 'neutral',
'neutral', 'positive', 'positive', 'neutral', 'neutral',
'positive', 'positive', 'positive', 'neutral', 'neutral',
'neutral'], dtype='<U8')

```

In [83]:

```

model2 = MultinomialNB()
model2.fit(xv_train,y_train)
predict2 = model2.predict(xv_train)
predict2

```

Out[83]:

```

array(['neutral', 'positive', 'neutral', 'negative', 'positive',
      'neutral', 'positive', 'neutral', 'neutral', 'neutral', 'positive',
      'positive', 'neutral', 'positive', 'neutral', 'neutral',
      'positive', 'negative', 'neutral', 'negative', 'neutral',
      'neutral', 'positive', 'neutral', 'negative', 'neutral', 'neutral',
      'negative', 'negative', 'neutral', 'positive', 'positive',
      'negative', 'neutral', 'neutral', 'neutral', 'neutral', 'negative',
      'neutral', 'negative', 'neutral', 'neutral', 'positive',
      'negative', 'positive', 'positive', 'positive', 'neutral',
      'positive', 'neutral', 'positive', 'negative', 'positive',
      'neutral', 'positive', 'neutral', 'neutral', 'positive', 'neutral',
      'neutral', 'positive', 'positive', 'negative', 'neutral',
      'positive', 'positive', 'negative', 'neutral', 'neutral',
      'negative', 'neutral', 'positive', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'positive', 'neutral', 'neutral',
      'negative', 'neutral', 'neutral', 'negative', 'negative',
      'negative', 'neutral', 'neutral', 'positive', 'neutral',
      'negative', 'negative', 'neutral', 'neutral', 'neutral',
      'negative', 'neutral', 'positive', 'neutral', 'positive',
      'positive', 'positive', 'positive', 'positive', 'negative',
      'positive', 'neutral', 'neutral', 'neutral', 'positive',
      'positive', 'neutral', 'neutral', 'neutral', 'negative',

```

[illegible]

```
print(accuracy_score(y_test,predict1))
0.608
```

```
print(accuracy_score(y_train,predict2))
0.946524064171123
```

BernoulliNB

```
model3 = BernoulliNB()
model3.fit(xv_train,y_train)
predict3 = model3.predict(xv_test)
predict3
```

```
array(['neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'positive',
      'neutral', 'positive', 'positive', 'neutral', 'neutral',
      'positive', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'positive', 'neutral', 'negative', 'positive', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'negative', 'neutral', 'positive', 'positive',
      'neutral', 'neutral', 'neutral', 'positive', 'neutral', 'positive',
      'neutral', 'neutral', 'positive', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'positive', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'negative', 'neutral',
      'neutral', 'neutral', 'negative', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'positive', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral', 'positive', 'neutral', 'positive',
      'positive', 'neutral', 'neutral', 'neutral', 'positive',
      'positive', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',
      'neutral', 'neutral', 'neutral'], dtype='<U8')
```

```
model4 = BernoulliNB()
model4.fit(xv_train,y_train)
predict4 = model4.predict(xv_train)
predict4
```

```
array(['neutral', 'positive', 'neutral', 'neutral', 'positive', 'neutral',  
      'positive', 'neutral', 'neutral', 'neutral', 'neutral', 'positive',  
      'neutral', 'positive', 'neutral', 'neutral', 'negative',  
      'negative', 'neutral', 'negative', 'neutral', 'neutral',  
      'positive', 'neutral', 'neutral', 'neutral', 'neutral', 'negative',  
      'neutral', 'neutral', 'neutral', 'positive', 'negative', 'neutral',
```

[illegible]


```
'negative', 'neutral', 'positive', 'negative', 'positive',  
'neutral', 'neutral', 'positive', 'neutral', 'negative', 'neutral',  
'neutral', 'neutral', 'positive', 'negative', 'neutral', 'neutral',  
'positive', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',  
'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'positive',  
'neutral', 'neutral', 'neutral', 'neutral', 'neutral', 'neutral',  
'neutral', 'positive', 'positive', 'neutral', 'positive',  
'neutral', 'positive', 'neutral', 'positive', 'neutral', 'neutral',  
'neutral', 'neutral', 'positive', 'neutral', 'neutral', 'neutral',  
'neutral', 'negative', 'positive', 'neutral', 'positive',  
'positive', 'neutral', 'neutral', 'positive', 'positive',  
'neutral', 'neutral', 'negative', 'neutral', 'neutral', 'neutral',  
'neutral', 'neutral', 'negative'], dtype='<U8')
```

In [95]:

```
print(accuracy_score(y_test, predict3))  
0.44
```

In [97]:

```
print(accuracy_score(y_train, predict4))  
0.8074866310160428
```

CONCLUSION

- This project successfully demonstrates how Natural Language Processing and Machine Learning can be applied to understand sentiments in social media posts. By converting unstructured textual data into meaningful insights, the system can automatically detect positive and negative emotions.
- The trained model can be used for:
 1. Monitoring brand reputation,
 2. MultinomialNB is the best model of the project
 3. Analyzing social trends in real time.
- Future improvements may include using deep learning models (e.g., LSTM, BERT) to handle context more a